

# **Advance Database Management**

## **FINAL PROJECT ASSIGNMENT**

**Name:** Fazeel Ahmed

**Matricola:** S5640427

**Group:** Single Member

Course: **ADVANCED DATA MANAGEMENT - 61884**

Professor: **Barbara Catania and Giovanna Guerrini**

## **Assignment Task 1**

### **1. Identify a workload in Natural Language**

Q1. Given a player identifier, determine the names of all stadiums in which the player has played at least once. **(Player)**

Q2. Given a player position and a specified market value, determine the names of all players satisfying those conditions. **(Player)**

Q3. Given a country name and a competition identifier, determine the types of all game events that occurred in the games of that competition. **(Competition)**

Q4. Given a player identifier and a specific year, determine all games played by that player during that year. **(Game)**

Q5. Given a specific year and a player name, determine the player's last season and the total number of minutes played during that year. **(Game)**

#### **Verification of the Constraints:**

| <b>Constraints</b>                               | <b>Result</b> | <b>Relevant queries</b>                 |
|--------------------------------------------------|---------------|-----------------------------------------|
| At least five structurally different operations  | Satisfied     | <b>Q1–Q5</b>                            |
| Three aggregation entities                       | Satisfied     | <b>(Player , Competition, and Game)</b> |
| At least three queries selecting on two entities | Satisfied     | <b>Q2, Q3, Q4, Q5</b>                   |
| At least two paths with length $\geq 2$          | Satisfied     | <b>Q1 and Q3</b>                        |
| Partition-key case (c)                           | Satisfied     | <b>Q4 and Q5</b>                        |

### **2. Use the aggregate-oriented design methodology (STEP 1-2-3) to design a set of logical aggregates for the given domain and the selected workload.**

#### **Step 1 (Query Modeling):**

##### **Q1:**

- E = Player
- LS = [Player(player\_id)\_!]
- LP = [Stadium(name)\_PiPi]

Q1 ( Player, [Player(player\_id)\_!], [Stadium(name)\_PiPi] )

**Q2:**

- E = Player
- LS = [Player(position)\_!, PlayerValuation(market\_value)\_OF]
- LP = [Player(name)\_!]

Q2 ( Player, [ Player(position)\_!, PlayerValuation(market\_value)\_OF], [ Player(name)\_! ] )

**Q3:**

- E = Competition
- LS = [Competition(competition\_id)\_!, Country(name)\_In]
- LP = [GameEvents(type)\_OH]

Q3 ( Competition, [ Competition(competition\_id)\_! , Country(name)\_In], [ GameEvents(type)\_OH ] )

**Q4:**

- E = Game
- LS = [Game(date)\_!, Player(player\_id)\_Pi]
- LP = [Game(game\_id, season)\_!]

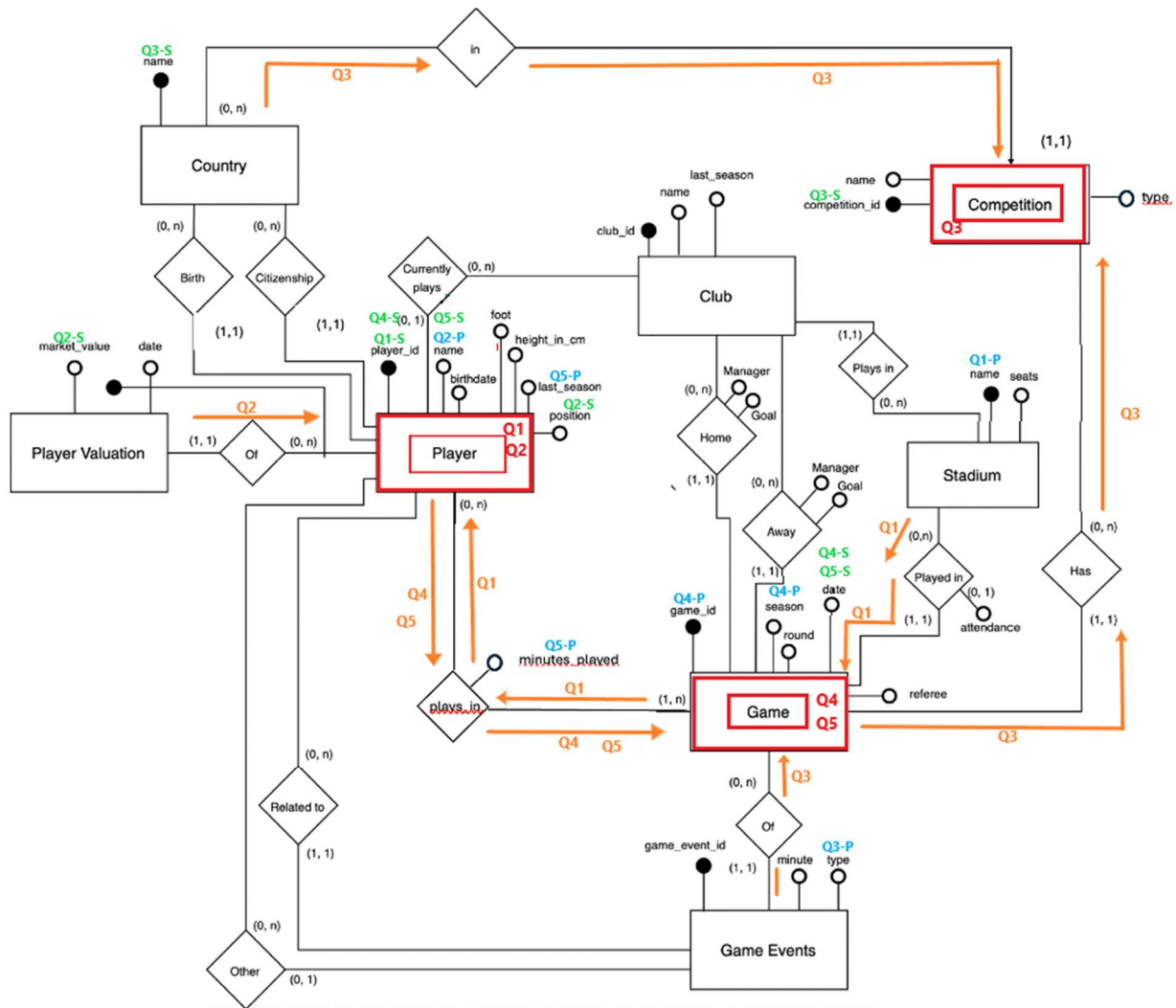
Q4 ( Game, [ Game(date)\_! , Player(player\_id)\_Pi ], [ Game(game\_id, season)\_! ] )

**Q5:**

- E = Game
- LS = [Game(date)\_!, Player(name)\_Pi]
- LP = [Player(last\_season)\_Pi(minutes\_played)]

Q5 ( Game, [ Game(date)\_!, Player(name)\_Pi ], [ Player(last\_season)\_Pi(minutes\_played) ] )

**Step 2 (Annotation of the ER Schema):**



### Step 3 (Translation into JSON Meta Notation):

#### Player Aggregate (Q1 and Q2):

Player =

```
{
  player_id, name, position,
  PlaterValuationOF: [ {marketvalue} ],
  plays_in:[ {Game: {game_id, name}}]
}
```

#### Competition Aggregate (Q3):

Competition =

```
{
  competition_id, country_name,
  Games: [ { Game: {game_id, game_event: [ {type} ] } } ]
}
```

**Game Aggregate (Q4 and Q5):**

```
Game =
{
  game_id, season, date,
  play_by:[ { player_id, name, last_season, minutes_played } ],
}
```

**3. For each aggregate-oriented schema and each associated query, specify the query type (either batch or random access query).**

- **Player Aggregate (Q1 and Q2):**

Random access query, because both queries contain selection entities (Player.player\_id, Player.position, and PlayerValuation.market\_value). Therefore, only a subset of the Player aggregates will be accessed, and no joining with other collections is required.

- **Competition Aggregate (Q3)**

Random access query (point query), because the query contains selection entities (Competition.competition\_id and Country.name). Therefore, only a subset of the Competition aggregates will be accessed, and no joining with other collections is required.

- **Game Aggregate (Q4 and Q5)**

Random access query, because both queries contain selection entities (Game.date, Player.player\_id, and Player.name). Therefore, only a subset of the Game aggregates will be accessed, and no joining with other collections is required.

All queries are Random Access Queries.

**4. For each designed aggregate-oriented schema, specify a suitable partition key and a suitable super-identifier for the corresponding collection.**

- **Player Aggregate (Q1 & Q2):**

Player = { player\_id<sup>P</sup>, name, position, PlayerValuationOF: [{market\_value}], plays\_in: [{Game: {game\_id, name}}] }

**Selection Attributes:**

**Q1** = {player\_id}

**Q2** = {position, market\_value}

**Case: (d)** – The two queries do not share any common selection attribute.

**Partition Key:** player\_id

**Super-Identifier:** player\_id

Q1 is favoured because it directly searches a player using player\_id. Since player\_id is already the identifier and partition key, no extension is required.

- **Competition Aggregate (Q3)**

Competition = { competition\_id<sup>p1</sup>, country\_name<sup>p2</sup>, Games: [ { Game: {game\_id, game\_event: [ {type} ] } } ] }

**Selection Attribute:** Q3 = {competition\_id, country\_name}

**Case:** (b) – All queries share the same selection attributes.

**Partition Key:** (competition\_id, country\_name)

**Super-Identifier:** (competition\_id, country\_name)

The original identifier competition\_id is extended with country\_name because the super-identifier must include the complete partition key.

- **Game Aggregate (Q4 & Q5)**

Game = { game\_id, season, date<sup>p</sup>, play\_by: [ { player\_id, name, last\_season, minutes\_played } ] }

**Selection Attributes:**

Q4 = {date, player\_id}

Q5 = {date, player\_name}

**Case:** (c) – Both queries share the common selection attribute date, but their complete selection sets are different.

**Partition Key:** date

**Super-Identifier:** (date, game\_id)

The original identifier game\_id is extended with date because the super-identifier must include the partition key.

## 5. For each designed aggregate-oriented schema and each associated query, specify if it can be executed efficiently on the collection.

- **Player Aggregate (Q1 and Q2):**

Q1: Efficient.

The query contains an equality condition on the complete partition key player\_id. Therefore, one node is accessed and the required aggregate is directly located.

Q2: Not efficient.

The query selects by position and market\_value, but the partition key is player\_id. Therefore, the system may need to access all nodes.

- **Competition Aggregate (Q3)**

Q3: Efficient.

The query contains equality conditions on both partition-key attributes, competition\_id and country\_name. Therefore, one node is accessed and the required aggregates are sequentially stored.

- **Game Aggregate (Q4 and Q5)**

Q4: Not fully efficient.

The condition on date identifies one node, but the query also contains a condition on player\_id. Therefore, the relevant Game aggregates may not be sequentially stored on that node.

Q5: Not fully efficient.

The condition on date identifies one node, but the query also filters by player\_name. Therefore, the relevant aggregates may not be sequentially stored on the node.

**6. For each designed aggregate-oriented schema, identify the indexes to be created to make the execution of each query associated with the collection efficient (if any).**

- **Player Aggregate (Q1 and Q2):**

A local ordered index must be created on the super-identifier: (player\_id)

No additional index is required for Q1 because it selects by the partition key.

To improve Q2, a secondary local index should be created on: (position, market\_value)

- **Competition Aggregate (Q3)**

No additional index is required because Q3 already uses all partition-key attributes and can be executed efficiently.

- **Game Aggregate (Q4 and Q5)**

A local ordered index must be created on the super-identifier, with the partition key first: (date, game\_id)

To improve Q4, create a secondary local index on: (date, player\_id)

To improve Q5, create a secondary local index on: (date, player\_name)

**DISCLOSURE:** I used a Large Language Model (LLM) to improve the wording of the natural language queries and to help generate and understand the partition key and super-identifier, as applying the partition key selection cases to the queries was sometimes confusing. The final submission reflects my own understanding after reviewing the lecture material.